Bài tập lớn - Level 1

Lập trình Thủ tục (Procedural programming)

Viện Trí tuệ Nhân tạo, Trường Đại học Công nghệ, ĐHQGHN

Ngày 17 tháng 3 năm 2026

Mục lục

I Giới thiệu (Introduction) 4

I.1 Tổng quan về trò chơi (Game Overview) 4

I.2 Luật kết thúc (End Rule) 5

I.3 Cấu trúc chương trình 5

II Mục tiêu học tập (Learning Objectives) 6

II.1 Sử dụng cấu trúc dữ liệu cơ bản 6

II.2 Thiết kế chương trình theo phong cách lập trình thủ tục 7

II.3 Xây dựng vòng lặp trò chơi 7

II.4 Cài đặt logic kiểm tra thắng 8

II.5 Làm quen với bot trò chơi đơn giản 8

III Luật trò chơi (Game Rules) 9

III.1 Kích thước bàn cờ 9

III.2 Ký hiệu trên bàn cờ 9

III.3 Mục tiêu chiến thắng (Goal) 9

III.4 Các hướng kiểm tra chuỗi 10

III.5 Luật kết thúc (End Rule) 11

III.5.1 NONE 11

III.5.2 OPEN_ONE 11
III.5.3 OPEN_TWO 12
III.6 Luật mặc định của trò chơi 12
III.7 Kết quả hòa (Draw) 13

IV Starter Code **13**
IV.1 Logger 13
IV.2 Argument Parser 14
IV.3 Run Configuration 14
IV.4 Judge Mode 15
IV.5 Game Helper 15
IV.6 Các phần cần cài đặt 16

V Cấu trúc chương trình (Program Structure) **16**
V.1 Kiến trúc tổng thể 17
V.2 Game Engine 17
V.3 Game Logic 18
V.4 Game Interaction 19
V.5 Game Renderer 20
V.6 Bot Module 21
V.7 Luồng thực thi và Tương tác giữa các module 22

VI Hướng dẫn triển khai (Implementation Guide) **23**
VI.1 Khai thác Starter Code 23
VI.2 Quản lý thông báo và Debugging 23
VI.3 Lộ trình triển khai khuyến nghị (Workflow) 24

VII Hướng dẫn thực nghiệm (Experiment Instructions) **25**
VII.1 Biên dịch chương trình 25
VII.2 Chạy chế độ tương tác (Interactive Mode) 26
VII.3 Chế độ Judge Mode 26
VII.4 Chạy Test Script (Grader) 26
VII.5 Ghi nhận kết quả vào báo cáo 26

VIII Yêu cầu báo cáo (Report Requirements) 27

VIII.1 Định dạng và Quy định chung 27

VIII.2 Cấu trúc nội dung (Outline) 28

IX Chính sách sử dụng công cụ AI (AI Usage Policy) 30

IX.1 Nguyên tắc chung 30

IX.2 Yêu cầu khai báo 30

IX.3 Những việc không được phép 31

IX.4 Mục tiêu của chính sách 31

X Nộp bài (Submission) 32

X.1 Các thành phần cần nộp 32

X.2 Cấu trúc thư mục 32

X.3 Định dạng nộp bài 33

X.4 Thời hạn nộp bài 33

X.5 Lưu ý 33

I. Giới thiệu (Introduction)

Trong bài tập lớn (programming assignment) này, các bạn sẽ xây dựng một chương trình trò chơi dựa trên *TicTacToe mở rộng* (*Generalized TicTacToe*) bằng ngôn ngữ lập trình `C++`.

Khác với phiên bản *TicTacToe cổ điển* với bàn cờ 3×3, chương trình trong bài tập này cho phép:

- **Cấu hình màn chơi:** thay đổi kích thước bàn cờ (`size`), thay đổi số quân liên tiếp cần đạt để thắng (`goal`),
- **Đa dạng luật chơi:** áp dụng nhiều *luật kết thúc* (*End Rule*) khác nhau,
- **Nhiều chế độ chơi:** hỗ trợ cả chế độ *người chơi với người chơi* (*Player vs Player*) và *người chơi với máy* (*Player vs Bot*).

Mục tiêu của bài tập là giúp các bạn làm quen với việc xây dựng một chương trình hoàn chỉnh với nhiều thành phần phối hợp với nhau, bao gồm:

- **Thực hành các nguyên lý lập trình**: sử dụng ngôn ngữ `C++` trong các bài toán như quản lý dữ liệu bằng *mảng hai chiều* (*2D Array*), xử lý *kiểm tra dữ liệu đầu vào* (*Input Validation*),
- **Áp dụng phong cách lập trình thủ tục (*Procedural Programming*):** thiết kế chương trình thành nhiều hàm (functions) nhỏ, mỗi hàm đảm nhiệm một nhiệm vụ cụ thể,
- **Thực hành lập trình trò chơi:** như xây dựng *vòng lặp trò chơi* (*Game Loop*), cài đặt *logic kiểm tra thắng* (*Win Checking Logic*),
- **Làm quen với khái niệm Trí tuệ Nhân tạo đơn giản:** phát triển một *bot đơn giản* (*Simple Game AI*).

Bài tập này được thiết kế theo hướng **tăng dần độ khó (progressive difficulty)**. Một số phần là bắt buộc, trong khi một số phần khác được khuyến khích để các bạn khám phá thêm. Điều này cho phép các bạn ở nhiều mức độ khác nhau đều có thể hoàn thành bài tập, đồng thời vẫn có cơ hội thử thách bản thân với các ý tưởng nâng cao. Phần tiếp theo sẽ mô tả tổng quan luật chơi của hệ thống.

I.1 Tổng quan về trò chơi (Game Overview)

Trò chơi diễn ra trên một bàn cờ hình vuông kích thước $N \times N$, `size` (N); trong đó hai người chơi lần lượt đặt ký hiệu của mình lên các ô trống.

- Người chơi thứ nhất sử dụng ký hiệu X.
- Người chơi thứ hai sử dụng ký hiệu O.

Mỗi lượt, người chơi chọn một ô trống trên bàn cờ và đặt ký hiệu của mình vào đó. Trò chơi kết thúc khi xảy ra một trong hai trường hợp sau:

- Một người chơi đạt được `goal` ký hiệu liên tiếp theo một trong bốn hướng:
  - ngang (*horizontal*),
  - dọc (*vertical*),
  - chéo chính (*main diagonal*),
  - chéo phụ (*anti-diagonal*).
- Bàn cờ đã được lấp đầy nhưng không có người chơi nào đạt điều kiện thắng (kết quả hòa – **draw**).

Ví dụ, nếu:

```
N = 5,  goal = 4
```

thì người chơi cần đạt được **4 ký hiệu liên tiếp** để thắng trò chơi.

I.2 Luật kết thúc (End Rule)

Ngoài điều kiện đạt đủ số quân liên tiếp, trò chơi còn áp dụng các *luật kết thúc* (*End Rule*) để xác định một chuỗi thắng có hợp lệ hay không. Ba luật được định nghĩa trong chương trình dưới dạng:

```cpp
enum class EndRule {
    NONE,
    OPEN_ONE,
    OPEN_TWO
};
```

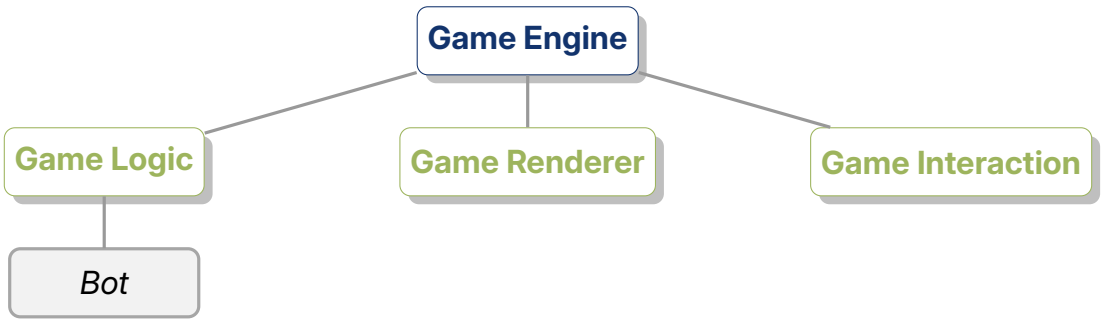
Trong bài tập này, trò chơi mặc định sử dụng luật:

```
EndRule::OPEN_TWO
```

Điều này có nghĩa là một chuỗi thắng chỉ hợp lệ nếu hai ô liền kề ở hai đầu chuỗi (nếu tồn tại) đều là ô trống. Chi tiết về các luật này sẽ được trình bày trong phần **Game Rules** của tài liệu.

I.3 Cấu trúc chương trình

Để giúp các bạn tập trung vào phần logic của trò chơi, một số thành phần của hệ thống đã được cung cấp sẵn dưới dạng **starter code** (file `starter_code.cpp`). Các bạn sẽ cần hoàn thiện các phần còn lại để tạo thành một chương trình hoàn chỉnh.



Hình 1: Cấu trúc chương trình

Chương trình được tổ chức thành nhiều nhóm chức năng chính:

- **Game Engine** – điều khiển vòng lặp trò chơi.
- **Game Logic** – xử lý trạng thái bàn cờ và điều kiện thắng.
- **Game Interaction** – nhận dữ liệu từ người chơi.
- **Game Renderer** – hiển thị trạng thái trò chơi.
- **Bot Module** – cài đặt chiến lược cho máy chơi.

Việc tách chương trình thành các module như trên giúp chương trình dễ hiểu hơn, dễ mở rộng hơn, và là một kỹ thuật phổ biến trong thiết kế phần mềm (**software design**).

Trong các phần tiếp theo của tài liệu, chúng ta sẽ mô tả chi tiết luật chơi, cấu trúc dữ liệu, và các thành phần mà các bạn cần cài đặt.

II. Mục tiêu học tập (Learning Objectives)

Bài tập này được thiết kế nhằm giúp các bạn củng cố và vận dụng các kiến thức đã học trong học phần lập trình, đồng thời làm quen với cách xây dựng một chương trình phần mềm hoàn chỉnh từ nhiều thành phần nhỏ. Sau khi hoàn thành bài tập, các bạn được kỳ vọng có thể đạt được các mục tiêu học tập sau.

II.1 Sử dụng cấu trúc dữ liệu cơ bản

Các bạn sẽ thực hành làm việc với *mảng hai chiều* (*2D Array*) để biểu diễn trạng thái của bàn cờ. Cụ thể, toàn bộ bàn cờ của trò chơi sẽ được lưu trữ trong một cấu trúc dữ liệu dạng:

```
char board[12][12];
```

Thông qua bài tập này, các bạn sẽ hiểu rõ hơn cách:

- truy cập và cập nhật phần tử trong mảng hai chiều,
- duyệt qua các hàng và cột của mảng,
- kiểm tra các mẫu (pattern) trong cấu trúc dữ liệu dạng lưới.

Những kỹ năng này rất quan trọng trong nhiều bài toán lập trình khác như xử lý ma trận, xử lý ảnh (*image processing*) hoặc xây dựng các trò chơi dạng lưới.

II.2 Thiết kế chương trình theo phong cách lập trình thủ tục

Trong bài tập này, các bạn sẽ áp dụng *phong cách lập trình thủ tục* (*Procedural Programming*) để tổ chức chương trình. Thay vì viết toàn bộ chương trình trong một khối mã lớn, các bạn sẽ chia chương trình thành nhiều hàm (*functions*) nhỏ, mỗi hàm đảm nhiệm một nhiệm vụ cụ thể.

Ví dụ, chương trình có thể được chia thành các nhóm hàm như:

- các hàm xử lý **logic trò chơi** (Game Logic),
- các hàm **tương tác với người chơi** (Game Interaction),
- các hàm **hiển thị bàn cờ** (Game Renderer),
- và các hàm điều khiển **vòng lặp trò chơi** (Game Engine).

Việc chia nhỏ chương trình thành các hàm giúp mã nguồn:

- dễ đọc hơn,
- dễ kiểm thử (*testing*) hơn,
- và dễ mở rộng (*extensible*) trong tương lai.

II.3 Xây dựng vòng lặp trò chơi

Một thành phần quan trọng của hầu hết các trò chơi là *vòng lặp trò chơi* (*Game Loop*). Trong vòng lặp này, chương trình sẽ liên tục thực hiện các bước:

1. hiển thị trạng thái hiện tại của trò chơi,
2. nhận hành động từ người chơi,
3. cập nhật trạng thái bàn cờ,
4. kiểm tra xem trò chơi đã kết thúc hay chưa.

Các bạn sẽ cài đặt cơ chế này thông qua các hàm như `startGame()`, `playGame()` và `endGame()`.

Thông qua đó, các bạn sẽ hiểu rõ hơn cách các chương trình tương tác (interactive programs) hoạt động trong thực tế.

II.4 Cài đặt logic kiểm tra thắng

Một phần quan trọng của bài tập là xây dựng thuật toán để kiểm tra điều kiện thắng của trò chơi. Cụ thể, các bạn cần cài đặt hàm `checkWin()` để xác định xem một người chơi đã đạt được `goal` ký hiệu liên tiếp hay chưa.

Thuật toán này cần kiểm tra các chuỗi ký hiệu theo bốn hướng khác nhau:

- ngang (*horizontal*),
- dọc (*vertical*),
- chéo chính (*main diagonal*),
- chéo phụ (*anti-diagonal*).

Ngoài ra, thuật toán cũng cần xem xét các *luật kết thúc* (*End Rule*) của trò chơi để xác định một chuỗi thắng có hợp lệ hay không.

Việc thiết kế thuật toán cho bài toán này giúp các bạn rèn luyện kỹ năng:

- phân tích bài toán,
- xây dựng thuật toán,
- và triển khai thuật toán bằng mã nguồn `C++`.

II.5 Làm quen với bot trò chơi đơn giản

Bên cạnh chế độ *Player vs Player*, trò chơi còn hỗ trợ chế độ *Player vs Bot*. Trong chế độ này, máy tính sẽ tự động chọn nước đi tiếp theo.

Ở mức cơ bản (`EASY`), các bạn sẽ cài đặt một bot đơn giản sử dụng chiến lược chọn nước đi ngẫu nhiên (*random move selection*). Mặc dù chiến lược này khá đơn giản, nó giúp các bạn hiểu được cách một chương trình có thể tự động đưa ra quyết định trong một môi trường trò chơi.

Đối với những các bạn muốn thử thách thêm, các mức độ bot cao hơn có thể được cài đặt bằng các chiến lược nâng cao hơn.

Thông qua việc hoàn thành bài tập này, các bạn không chỉ củng cố kỹ năng lập trình mà còn hiểu rõ hơn cách một hệ thống phần mềm nhỏ được thiết kế, triển khai và kiểm thử trong thực tế.

III. Luật trò chơi (Game Rules)

Phần này mô tả chi tiết các quy tắc của trò chơi *TicTacToe mở rộng* (*Generalized TicTacToe*) được sử dụng trong bài tập. Các bạn cần hiểu rõ các quy tắc này để có thể cài đặt đúng logic của chương trình.

III.1 Kích thước bàn cờ

Trò chơi diễn ra trên một bàn cờ hình vuông có kích thước:

$$\texttt{size} \times \texttt{size}$$

Trong đó giá trị của `size` phải thỏa điều kiện:

$$3 \leq \texttt{size} \leq 12$$

Bàn cờ được biểu diễn trong chương trình bằng một mảng hai chiều:

```
char board[12][12];
```

Mỗi phần tử của mảng tương ứng với một ô trên bàn cờ.

III.2 Ký hiệu trên bàn cờ

Mỗi ô trên bàn cờ có thể chứa một trong ba giá trị sau:

`'-'`	ô trống
`'X'`	quân của người chơi 1
`'O'`	quân của người chơi 2

Ban đầu, toàn bộ bàn cờ được khởi tạo với ký hiệu:

`'-'`

Mỗi lượt, người chơi sẽ chọn một ô trống và đặt ký hiệu của mình vào ô đó.

III.3 Mục tiêu chiến thắng (Goal)

Một người chơi thắng trò chơi nếu họ đạt được một chuỗi gồm `goal` ký hiệu liên tiếp của mình.

Giá trị của `goal` được nhập khi bắt đầu trò chơi và phải thỏa điều kiện:

$$3 \leq \texttt{goal} \leq \texttt{size}$$

Điều này cho phép trò chơi linh hoạt hơn so với TicTacToe cổ điển.

Ví dụ:

```
size = 8,  goal = 5
```

Trong trường hợp này, người chơi cần đạt được 5 ký hiệu liên tiếp để thắng.

III.4 Các hướng kiểm tra chuỗi

Chuỗi ký hiệu liên tiếp có thể xuất hiện theo bốn hướng khác nhau:

- ngang (*horizontal*)
- dọc (*vertical*)
- chéo chính (*main diagonal*)
- chéo phụ (*anti-diagonal*)

Ví dụ, với `size = 6, goal = 4`, một chuỗi thắng theo hướng ngang có thể có dạng:

```
- - - - - -
- - - - - -
- X X X X -
- - - - - -
- - - - - -
- - - - - -
```

Trong khi một chuỗi thắng theo đường chéo có thể có dạng:

```
- - - - - -
- X - - - -
- - X - - -
- - - X - -
- - - - X -
- - - - - -
```

Các bạn cần cài đặt thuật toán để phát hiện các chuỗi như vậy trong hàm:

```
checkWin(...)
```

III.5 Luật kết thúc (End Rule)

Ngoài việc đạt đủ `goal` ký hiệu liên tiếp, trò chơi còn áp dụng các *luật kết thúc* (*End Rule*) để xác định xem chuỗi đó có hợp lệ hay không.

Ba luật được định nghĩa trong chương trình như sau:

```
enum class EndRule {
    NONE,
    OPEN_ONE,
    OPEN_TWO
};
```

Ý nghĩa của từng luật được mô tả dưới đây.

III.5.1 NONE

Với luật `EndRule::NONE`, chỉ cần đạt đủ `goal` ký hiệu liên tiếp là được tính là thắng.

Ví dụ:

```
- - - - - -
O X X X X O
- - - - - -
```

Chuỗi XXXX vẫn được tính là chiến thắng mặc dù **hai đầu của chuỗi không phải là ô trống**. Thực ra trong trường hợp này khi người chơi X đạt được 4 ký tự liên tiếp thì đã chiến thắng ngay, trên chỉ là một ví dụ minh hoạ.

III.5.2 OPEN_ONE

Với luật `EndRule::OPEN_ONE`, chuỗi thắng phải có ít nhất một đầu mở (tức là một ô trống hoặc cùng ký tự chơi ở một trong hai đầu của chuỗi).

Ví dụ hợp lệ:

```
- - - - - -
- X X X X O
- - - - - -
```

Ví dụ không hợp lệ:

```
- - - - - -
O X X X X O
- - - - - -
```

Trong trường hợp thứ hai, cả hai đầu của chuỗi đều bị chặn bởi quân của đối thủ.

III.5.3 OPEN_TWO

Với luật `EndRule::OPEN_TWO`, chuỗi thắng phải có hai đầu mở (hai ô trống hoặc cùng ký tự chơi ở hai đầu của chuỗi).

Ví dụ hợp lệ:

```
- - - - - -
- X X X X -
- - - - - -
```

Ví dụ không hợp lệ:

```
- - - - - -
O X X X X -
- - - - - -
```

Trong ví dụ trên, một đầu của chuỗi bị chặn nên chuỗi không được tính là chiến thắng.

Ví dụ thắng hợp lệ:

```
- - - - - -
O X X X X X
- - - - - -
```

Trong ví dụ trên, nếu nhìn vào chuỗi bên phải cùng sẽ thấy 4 ký tự `X` liên tiếp không bị chặn 2 đầu (đầu bên trái là ký tự `X` còn đầu bên phải là ranh giới bàn chơi).

III.6 Luật mặc định của trò chơi

Trong bài tập này, trò chơi sẽ sử dụng luật mặc định:

```
EndRule::OPEN_TWO
```

Điều này có nghĩa là một chuỗi thắng chỉ hợp lệ nếu hai ô liền kề ở hai đầu của chuỗi (nếu tồn tại) đều mở (không phải ký tự của đối thủ).

Các bạn cần đảm bảo rằng hàm `checkWin()` kiểm tra đúng điều kiện này.

III.7 Kết quả hòa (Draw)

Trò chơi được xem là hòa (*draw*) nếu:

- toàn bộ bàn cờ đã được lấp đầy,
- và không có người chơi nào đạt điều kiện chiến thắng.

Trong trường hợp này, chương trình cần kết thúc trò chơi và thông báo kết quả hòa cho người chơi.

IV. Starter Code

Để giúp các bạn tập trung vào việc cài đặt logic của trò chơi, một bộ **starter code** đã được cung cấp trong file:

`starter_code.cpp`

Starter code này đã cài đặt sẵn một số thành phần cơ bản của hệ thống. Các bạn cần đọc và hiểu cấu trúc của chương trình trước khi bắt đầu viết các phần còn thiếu.

Mục tiêu của starter code là:

- cung cấp khung chương trình (*skeleton*) cho toàn bộ hệ thống,
- chuẩn hóa cách ghi log và cách chạy chương trình,
- giúp việc kiểm thử tự động (*automated testing*) trở nên dễ dàng hơn.

Trong phần này, chúng ta sẽ mô tả các thành phần chính của starter code.

IV.1 Logger

Mã nguồn starter code cung cấp một hệ thống ghi log đơn giản thông qua module **GameLogger**. Module này được sử dụng để in thông tin ra màn hình hoặc ghi vào file log.

Ví dụ sử dụng:

```
GameLogger::log("Game started");
```

Các bạn **bắt buộc phải sử dụng** hệ thống logger này thay vì gọi trực tiếp:

```
std::cout
```

Việc sử dụng logger giúp chương trình:

- dễ theo dõi hoạt động của hệ thống,
- dễ kiểm tra kết quả trong chế độ *Judge Mode*,
- và giúp giảng viên dễ dàng debug khi chấm bài.

IV.2 Argument Parser

Mã nguồn của starter code đã cung cấp sẵn một bộ phân tích tham số dòng lệnh (*argument parser*). Thành phần này cho phép chương trình nhận các tham số khi chạy từ terminal.

Ví dụ:

```
./game --judge
```

Một số tham số quan trọng bao gồm:

- `--judge` : chạy chương trình ở chế độ chấm tự động (*Judge Mode*)
- `--input` : đọc dữ liệu đầu vào từ file
- `--log` : ghi log ra file

Các bạn **không cần cài đặt lại** argument parser. Phần này đã được cung cấp đầy đủ trong starter code.

IV.3 Run Configuration

Starter code sử dụng một cấu trúc dữ liệu để lưu trữ các cấu hình khi chạy chương trình. Cấu trúc này thường được gọi là **Run Configuration**.

Ví dụ:

```
struct RunConfig {
    bool judgeMode;
    std::string inputFile;
    std::string logFile;
};
```

Thông tin trong cấu trúc này sẽ được sử dụng để quyết định cách chương trình hoạt động, ví dụ:

- chạy tương tác với người chơi,
- hoặc chạy ở chế độ chấm tự động.

Các bạn chỉ cần sử dụng các giá trị trong cấu trúc này, không cần thay đổi cách nó được tạo ra.

IV.4 Judge Mode

Starter code hỗ trợ một chế độ đặc biệt gọi là **Judge Mode**. Chế độ này được sử dụng để kiểm thử tự động chương trình.

Khi chương trình được chạy với tham số:

```
--judge
```

chương trình sẽ đọc dữ liệu đầu vào từ file và in ra kết quả theo một định dạng cố định.

Định dạng kết quả phải là:

```
<winner> <turns>
```

Trong đó:

- `winner` là người thắng cuộc
- `turns` là số lượt chơi đã diễn ra

Ví dụ:

```
0 7
```

hoặc

```
-1 9
```

Các bạn cần đảm bảo rằng chương trình của mình in ra đúng định dạng này khi chạy trong *Judge Mode*.

IV.5 Game Helper

Bên cạnh đó, starter code cung cấp một công cụ hỗ trợ quan trọng là hàm mẫu (template function) với tên là `measureExecutionTime()`. Hàm này giúp các bạn đo lường chính xác thời gian thực thi của một hàm bất kỳ trong chương trình.

Để sử dụng, các bạn gọi hàm theo cấu trúc:

```
measureExecutionTime(
    "botMove",              // label - func name
    [&]() {
        return botMove();   // func()
    },
    true                    // enabled (true if using logger)
);
```

Các đặc điểm chính của công cụ này bao gồm:

- **Độ chính xác cao**: Sử dụng `high_resolution_clock` để đo thời gian thực tế.
- **Tính linh hoạt**: Tự động nhận diện kiểu trả về (void hoặc có giá trị) của hàm được đo.
- **Tích hợp Logger**: Kết quả đo được sẽ được ghi lại trong *GameLogger* ở mức độ `DEBUG` nếu tham số `enabled` được bật.

Việc sử dụng công cụ này rất hữu ích khi các bạn cần tối ưu hóa hiệu suất của **Bot Module** hoặc kiểm tra các đoạn code xử lý nặng. Các bạn **không cần thay đổi** nội dung của hàm helper này.

IV.6 Các phần cần cài đặt

Mặc dù starter code đã cung cấp các thành phần cơ bản, các bạn cần tự hoàn thiện logic trò chơi thông qua việc cài đặt: **Game Engine** (điều khiển vòng lặp), **Game Logic** (xử lý trạng thái và kiểm tra thắng), **Game Interaction** (nhận dữ liệu), **Game Renderer** (hiển thị) và **Bot Module** (mức độ cơ bản). Chi tiết về các hàm này sẽ được trình bày trong phần tiếp theo của tài liệu.

V. Cấu trúc chương trình (Program Structure)

Như đã trình bày trong phần trước, chương trình được tổ chức thành nhiều thành phần khác nhau để tách biệt các trách nhiệm (*separation of concerns*). Việc tổ chức chương trình theo cấu trúc rõ ràng giúp mã nguồn dễ hiểu, dễ bảo trì và dễ mở rộng.

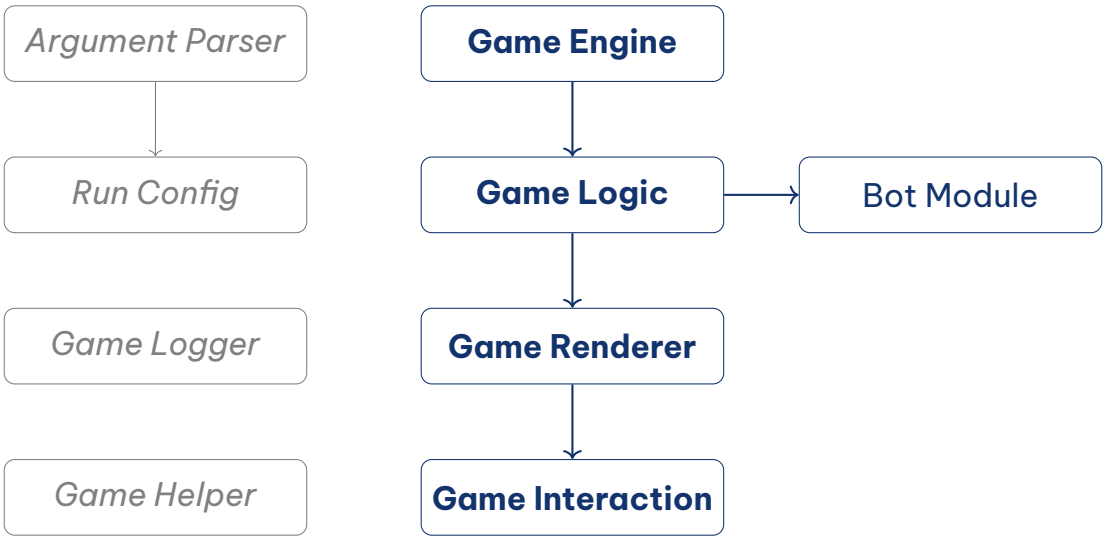
Trong bài tập này, hệ thống trò chơi được chia thành năm module chính:

- **Game Engine** – điều khiển vòng lặp của trò chơi
- **Game Logic** – quản lý trạng thái bàn cờ và kiểm tra điều kiện thắng
- **Game Interaction** – nhận dữ liệu đầu vào từ người chơi
- **Game Renderer** – hiển thị bàn cờ và thông tin trò chơi
- **Bot Module** – cài đặt chiến lược cho máy chơi

Các module này tương tác với nhau để tạo thành một chương trình hoàn chỉnh.

V.1 Kiến trúc tổng thể

Hình dưới đây minh họa cấu trúc tổng thể của hệ thống trò chơi.



Hình 2: Cấu trúc tổng thể của trò chơi

Trong kiến trúc này, **Game Engine** đóng vai trò là thành phần trung tâm. Nó điều khiển luồng hoạt động của trò chơi và gọi các module khác khi cần thiết.

V.2 Game Engine

Game Engine đóng vai trò trung tâm trong việc điều phối các module khác và kiểm soát vòng đời của trò chơi (*game lifecycle*). Để quản lý dữ liệu xuyên suốt quá trình này, starter code định nghĩa hai cấu trúc quan trọng:

```
struct GameSetup {
    char board[BOARD_N_MAX][BOARD_N_MAX];
    int size;
    int goal;
    GameMode mode;
    BotLevel levels[2];
};

struct GameResult {
    int winner; // 0: Player 1, 1: Player 2, -1: Draw
    bool isBot;
    int turns;
};
```

- **GameSetup**: Lưu trữ cấu hình và trạng thái hiện tại (bàn cờ `board`, kích thước `size`, điều kiện thắng `goal`, chế độ chơi `mode` và cấp độ bot `levels`).
- **GameResult**: Lưu trữ kết quả cuối cùng sau khi trận đấu kết thúc (chỉ số người

thắng `winner`, tổng số lượt đi `turns` và xác nhận người thắng có phải là bot hay không `isBot`).

Module này bao gồm ba hàm chính mà các bạn cần hoàn thiện:

```
\\Game Engine

void startGame(const RunConfig& config, GameSetup& gameSetup);

GameResult playGame(const RunConfig& config, GameSetup& gameSetup);

void endGame(const RunConfig& config, GameSetup& gameSetup, GameResult&
 gameResult);
```

- `startGame(config, gameSetup)`: Khởi tạo cấu hình trò chơi dựa trên lựa chọn của người dùng (kích thước bàn cờ, chế độ chơi PVP/PVE/EVE, cấp độ bot) và thiết lập trạng thái bàn cờ ban đầu.
- `playGame(config, gameSetup)`: Thực hiện vòng lặp chính (*main game loop*). Hàm này xác định lượt đi giữa người và bot, gọi các hàm từ module *Interaction* hoặc *Bot*, sau đó phối hợp với *Game Logic* để cập nhật bàn cờ và kiểm tra điều kiện kết thúc.
- `endGame(config, gameSetup, gameResult)`: Tổng kết và hiển thị kết quả. Tùy vào chế độ chạy (*Interactive* hoặc *Judge*) mà hàm sẽ trình bày thông tin thắng/thua hoặc in ra định dạng tối giản để hệ thống chấm điểm tự động.

Kết quả của quá trình thực thi được lưu trữ trong cấu trúc `GameResult`, bao gồm thông tin về người chiến thắng, tổng số lượt đi và xác nhận liệu người thắng có phải là bot hay không.

V.3 Game Logic

Game Logic là module thực thi các quy tắc cốt lõi của trò chơi. Khác với *Game Engine* đóng vai trò điều phối thời điểm hành động, module này tập trung hoàn toàn vào việc trả lời câu hỏi: *"Hành động đó có đúng luật không?"* và *"Trạng thái hiện tại của bàn cờ là gì?"*.

Các bạn cần cài đặt các hàm xương sống sau để đảm bảo trò chơi vận hành đúng quy tắc:

```
// Game Logic

void initBoard(char board[][BOARD_N_MAX], const int size);

bool isValidMove(const char board[][BOARD_N_MAX], const int size, const
 int row, const int col);
void makeMove(char board[][BOARD_N_MAX], const int row, const int col,
const char symbol);
```

```

bool isEmptyCell(char board[][BOARD_N_MAX], int size, int x, int y);

bool checkWin(char board[][BOARD_N_MAX], const int size, const char symbol, const int goal, EndRule rule = EndRule::OPEN_TWO);

bool checkDraw(char board[][BOARD_N_MAX], const int size);
```

- `initBoard(board, size)`: Khởi tạo tất cả các ô trên bàn cờ về ký tự trống `'-'`.
- `isValidMove(board, size, row, col)`: Kiểm tra nước đi có hợp lệ không? (nằm trong phạm vi bàn cờ và ô chọn có đang trống hay không).
- `isEmptyCell(board, size, x, y)`: Kiểm tra ô ở vị trí `(x, y)` có trống không?
- `makeMove(board, row, col, symbol)`: Cập nhật ký hiệu của người chơi (`'X'` hoặc `'O'`) vào vị trí tương ứng trên bàn cờ.
- `checkWin(board, size, symbol, goal, rule)`: Quét bàn cờ theo 4 hướng (ngang, dọc, chéo chính, chéo phụ) để tìm chuỗi ký hiệu liên tiếp đạt độ dài `goal`.
- `checkDraw(board, size)`: Xác định trạng thái hòa khi bàn cờ đã đầy (*full*) nhưng không có người chiến thắng.

Việc tách biệt logic này giúp mã nguồn dễ bảo trì và cho phép mở rộng các quy tắc nâng cao (như luật chặn hai đầu `EndRule`) mà không làm ảnh hưởng đến luồng điều khiển của Engine.

V.4 Game Interaction

Game Interaction chịu trách nhiệm quản lý toàn bộ quá trình nhận và phân tích dữ liệu đầu vào (*input handling*). Module này đóng vai trò là lớp đệm giữa người dùng (hoặc file dữ liệu) và hệ thống, đảm bảo dữ liệu thô được chuẩn hóa trước khi đưa vào xử lý.

Một nguyên tắc thiết kế quan trọng là module này **chỉ** xử lý định dạng nhập xuất và không can thiệp vào luật chơi (việc kiểm tra nước đi có hợp lệ trên bàn cờ hay không thuộc về *Game Logic*).

Các nhóm hàm chính các bạn cần hoàn thiện bao gồm:

```
// Game Interaction

bool validateInput(std::string& input);
bool getInput(int* val);

bool selectSize(int* size);
```

```
bool selectGoal(int* goal, const int size);
bool selectGameMode(GameMode* mode);
bool selectBotLevel(BotLevel* levels, const int index);

bool getPlayerMove(int* row, int* col);
```

- **Nhóm phân tách dữ liệu**: `validateInput` và `getInput` là hai hàm giúp đọc chuỗi từ `std::cin`, kiểm tra tính hợp lệ của ký tự số và chuyển đổi sang kiểu nguyên (`int`) một cách an toàn.
- **Nhóm thiết lập trò chơi**: Các hàm `selectSize`, `selectGoal`, `selectGameMode` và `selectBotLevel` điều hướng người dùng lựa chọn cấu hình bàn cờ, điều kiện thắng và độ khó của Bot.
- **Nhóm tương tác trong trận**: Hàm `getPlayerMove` nhận tọa độ nước đi (dạng `row col`) từ người chơi. Lưu ý tọa độ được tính theo chỉ số bắt đầu từ 0 (*zero-based*).

Nếu người dùng nhập sai định dạng hoặc giá trị nằm ngoài phạm vi cho phép, các hàm này sẽ trả về `false` để *Game Engine* có thể yêu cầu nhập lại thay vì làm chương trình bị dừng đột ngột.

Ngoài ra, starter code cung cấp sẵn hai hàm hệ thống là `initInteraction` và `closeInteraction` như sau:

```
// Given
std::streambuf* initInteraction(const RunConfig& config);
void closeInteraction(std::streambuf* cin_backup);
```

Các hàm này đảm nhận việc cấu hình luồng dữ liệu (*stream redirection*), cho phép chương trình tự động chuyển đổi giữa việc nhận dữ liệu trực tiếp từ bàn phím hoặc đọc từ file cấu hình thông qua tham số `--input`. Điều này giúp hệ thống có thể chạy kiểm thử tự động (*Judge Mode*) một cách linh hoạt mà không cần thay đổi mã nguồn xử lý nhập xuất bên trong.

V.5 Game Renderer

Game Renderer chịu trách nhiệm toàn bộ về việc hiển thị thông tin và giao diện người dùng trên thiết bị xuất (*console*). Module này hoạt động theo nguyên tắc **chỉ xuất dữ liệu** (output-only), tuyệt đối không can thiệp vào việc xử lý luật chơi hay thay đổi trạng thái bàn cờ.

Starter code cung cấp sẵn hàm `clearScreen()` sử dụng các mã điều khiển ANSI để làm mới màn hình terminal. Các bạn có trách nhiệm hoàn thiện các hàm hiển thị sau:

```
// Game renderer

void clearScreen(); // Given
```

```

void showSelectMenu(SelectType selectType);

void displayBoard(const char board[][BOARD_N_MAX], const int size);

void showMove(const int row, const int col);
void showInvalidMove();
void showPlayer(const int player, const bool is_bot);

void showResult(const int winner, const bool is_bot);
void printResult(const GameResult& gameResult);
```

- `showSelectMenu(selectType)`: Hiển thị các loại menu tương ứng với từng giai đoạn (Tiêu đề, chọn kích thước, chọn chế độ chơi, chọn độ khó của Bot...).
- `displayBoard(board, size)`: Vẽ bàn cờ ra màn hình một cách trực quan, bao gồm các chỉ số hàng/cột và các ký hiệu `'X'`, `'O'` hoặc `'-'`.
- `showPlayer` và `showMove`: Thông báo lượt đi hiện tại (người hay Bot) và vị trí nước đi vừa được thực hiện.
- `showResult` và `printResult`: Hiển thị thông báo kết thúc trận đấu. Trong đó, `printResult` được thiết kế riêng cho chế độ không tương tác (*non-interactive mode*) để in ra kết quả tối giản cho hệ thống chấm điểm.

Việc thiết kế tốt module này sẽ giúp giao diện trò chơi trở nên rõ ràng và chuyên nghiệp hơn, hỗ trợ đắc lực cho việc theo dõi diễn biến trận đấu.

V.6 Bot Module

Bot Module chịu trách nhiệm cài đặt các chiến lược trí tuệ nhân tạo (*AI strategies*) khi trò chơi chạy ở chế độ *Player vs Bot* hoặc *Bot vs Bot*. Module này tập trung vào việc tìm kiếm và trả về tọa độ nước đi tối ưu dựa trên trạng thái hiện tại của bàn cờ.

Các bạn sẽ thực hiện cài đặt thông qua hàm điều phối chính là `botMove`, với ba cấp độ thử thách khác nhau:

```
// Bot Move Logic
pII botMove(char board[][BOARD_N_MAX], const int size, const int goal,
const char symbol, const BotLevel level)

// Level 1 - EASY
pII random_pick(char board[][BOARD_N_MAX], const int size); // Required

// Level 2 - MEDIUM
pII simple_heuristic(char board[][BOARD_N_MAX], const int size, const
int goal, const char botSymbol, const char playerSymbol);

// Level 3 - HARD
```

```
pII hard_level(char board[][BOARD_N_MAX], const int size, const int goal, const char botSymbol, const char playerSymbol);
```

- **EASY (Bắt buộc)**: Cài đặt hàm `random_pick`. Bot thực hiện quét toàn bộ bàn cờ để thu thập danh sách các ô còn trống, sau đó chọn ngẫu nhiên một vị trí để đánh. Đây là yêu cầu tối thiểu để hoàn thành module.
- **MEDIUM (Khuyến khích)**: Cài đặt hàm `simple_heuristic`. Tại cấp độ này, bot không còn chọn ngẫu nhiên mà bắt đầu biết phân tích: ưu tiên thực hiện nước đi để thắng ngay lập tức hoặc ngăn chặn đối thủ khi họ sắp đạt đủ chuỗi `goal`.
- **HARD (Mở rộng/Sáng tạo)**: Cài đặt hàm `hard_level`. Đây là phần thử thách cộng điểm, không giới hạn thuật toán. Các bạn có thể tự do áp dụng các kỹ thuật như duyệt cây quyết định, hàm lượng giá (*evaluation function*) hoặc nhận diện mẫu hình (*pattern recognition*).

Lưu ý: Tất cả các hàm tìm kiếm nước đi đều phải trả về kiểu dữ liệu `pII` (tương đương `std::pair<int, int>`) đại diện cho tọa độ `row` và `col`.

V.7 Luồng thực thi và Tương tác giữa các module

Chương trình được thiết kế theo mô hình điều khiển trung tâm, trong đó hàm `main` đóng vai trò khởi tạo và kết nối các thành phần lớn. Quá trình vận hành diễn ra theo trình tự chặt chẽ như sau:

1. **Khởi tạo hệ thống**: Đầu tiên, *Argument Parser* phân tích tham số dòng lệnh để thiết lập `RunConfig`. Dựa vào đó, *Game Logger* và *Game Interaction* được kích hoạt để sẵn sàng ghi nhật ký và điều hướng luồng dữ liệu (từ bàn phím hoặc file).
2. **Thiết lập trò chơi**: Trong phần này, *Game Engine* gọi hàm `startGame` để chuẩn bị cấu trúc `GameSetup`. Tại đây, các module *Renderer* và *Interaction* phối hợp để hiển thị menu và nhận tùy chỉnh từ người dùng (kích thước bàn cờ, chế độ chơi, độ khó của Bot).
3. **Vòng lặp điều phối (Core Loop)**: Hàm `playGame` trong *Game Engine* sẽ quản lý vòng lặp chính của trò chơi:
   - Gọi *Game Renderer* để hiển thị trạng thái bàn cờ hiện tại.
   - Dựa vào lượt đi, gọi *Game Interaction* (nhận nước đi từ người) hoặc *Bot Module* (sinh nước đi tự động).
   - Chuyển tọa độ nước đi cho *Game Logic* để kiểm tra tính hợp lệ và cập nhật bàn cờ.
   - Sử dụng *Game Logic* để xác định điều kiện thắng (`checkWin`) hoặc hòa (`checkDraw`).

4. **Kết thúc và Tổng kết**: Khi vòng lặp kết thúc, thông tin kết quả được đóng gói vào `GameResult`. *Game Engine* gọi `endGame` để hiển thị kết quả cuối cùng qua *Renderer* và ghi lại lịch sử trận đấu thông qua *Logger*.
5. **Giải phóng tài nguyên**: Cuối cùng, *Game Interaction* trả lại luồng nhập xuất chuẩn và *Game Logger* đóng các tệp nhật ký để kết thúc chương trình an toàn.

Cách tổ chức này không chỉ giúp mỗi module tập trung vào một nhiệm vụ cụ thể (*Single Responsibility*) mà còn cho phép dễ dàng kiểm thử từng thành phần độc lập trong quá trình phát triển.

Trong phần tiếp theo của tài liệu, chúng ta sẽ mô tả chi tiết các hàm mà các bạn cần cài đặt để hoàn thiện chương trình.

VI. Hướng dẫn triển khai (Implementation Guide)

Phần này cung cấp định hướng tổng quát để các bạn triển khai chương trình. Các bạn cần đọc kỹ **starter code** đã được cung cấp và hoàn thiện các thành phần còn thiếu theo kiến trúc đã mô tả.

VI.1 Khai thác Starter Code

Toàn bộ mã nguồn mẫu đã được cấu trúc dưới dạng các **Skeleton Functions** (hàm khung). Trước khi bắt tay vào viết code, các bạn cần lưu ý:

- **Đọc kỹ Docstring**: Mỗi hàm đều đi kèm với phần chú thích chi tiết về: *Trách nhiệm (Responsibilities), Tham số (Parameters)* và *Giá trị trả về (Return)*. Đây là tài liệu hướng dẫn trực tiếp nhất để bạn hiểu logic cần cài đặt.
- **Tuân thủ Prototype**: Không thay đổi chữ ký hàm (tên hàm, kiểu trả về, tham số) vì các module đã được liên kết sẵn với nhau trong hàm `main`.
- **Gợi ý triển khai (TODO)**: Bên trong thân hàm, các bước thực hiện đã được liệt kê dưới dạng comment `// TODO`. Các bạn nên bám sát trình tự này để đảm bảo không bỏ sót các điều kiện biên.

VI.2 Quản lý thông báo và Debugging

Trong quá trình triển khai, các bạn được yêu cầu hạn chế tối đa việc sử dụng trực tiếp `std::cout` để in thông tin kiểm thử. Thay vào đó, hãy sử dụng module **GameLogger** được cung cấp sẵn để quản lý luồng thông tin một cách chuyên nghiệp:

- **Mức độ INFO (`GameLogger::Level::INFO`)**: Sử dụng để ghi lại các cột mốc quan trọng trong luồng chạy của chương trình (ví dụ: "Bắt đầu lượt đi của Bot", "Khởi tạo bàn cờ thành công"). Ví dụ sử dụng:

```
GameLogger::log(
    "Player X moved to (1,2)", GameLogger::Level::INFO
);
```

- **Mức độ DEBUG (`GameLogger::Level::DEBUG`)**: Sử dụng để in các giá trị biến, trạng thái trung gian hoặc kết quả đo thời gian từ `measureExecutionTime`. Các thông tin này sẽ giúp bạn tìm lỗi mà không làm bẩn giao diện hiển thị của người dùng. Ví dụ như trong `GameHelper` đã có sử dụng:

```
if (enabled) {
    // msg create
    std::chrono::duration<double> duration = end - start;
    std::stringstream msg;
    msg << "execution time of " << "[" << label << "()]" << " = "
    << duration.count() << "s";

    // debug log
    GameLogger::log(msg.str(), GameLogger::Level::DEBUG);
}
```

Việc sử dụng `GameLogger` rất dễ dàng nhờ cơ chế bật/tắt toàn bộ thông báo debug hoặc chuyển hướng nhật ký ra file `log.txt` chỉ bằng cách thay đổi cấu hình trong `RunConfig`.

Một số lưu ý khi sử dụng Logger:

- **Tính tối giản**: Nội dung log không nên quá phức tạp, tránh gây nhiễu khi theo dõi hệ thống.
- **Sự kiện trọng tâm**: Tập trung vào các sự kiện quan trọng như bắt đầu lượt đi, thay đổi trạng thái bàn cờ hoặc kết quả kiểm tra thắng thua.
- **Tư duy lập trình**: Không có một quy tắc cứng nhắc hay lời giải "đúng 100%" cho việc đặt log ở đâu, nhưng việc sử dụng Logger một cách khoa học là một **good practice** (thói quen tốt) giúp bạn kiểm soát mã nguồn hiệu quả hơn.

VI.3 Lộ trình triển khai khuyến nghị (Workflow)

Để xây dựng trò chơi một cách mạch lạc và dễ dàng kiểm soát lỗi, các bạn nên triển khai các module theo trình tự sau:

1. **Game Interaction (Giao tiếp)**: Ưu tiên hoàn thiện các hàm nhập liệu cơ bản như `validateInput`, `selectSize` và `selectGameMode`. Đây là bước đầu tiên để chương trình có thể hiểu được ý định của người chơi.

2. **Game Renderer (Hiển thị)**: Cài đặt `displayBoard` và các hàm thông báo. Khi kết hợp với module Interaction, bạn đã có thể thấy được khung giao diện và phản hồi từ chương trình ngay cả khi chưa có logic xử lý.
3. **Game Engine (Điều phối)**: Kết nối hai module trên vào vòng lặp `playGame`. Ở bước này, bạn có thể chạy thử luồng di chuyển giữa các lượt đi, dù bàn cờ chưa thực sự cập nhật hay kiểm tra thắng thua.
4. **Game Logic (Cốt lõi)**: Triển khai `makeMove`, `checkWin` và `checkDraw`. Đây là "bộ não" của trò chơi. Sau khi hoàn thành bước này, trò chơi của bạn đã có thể vận hành đầy đủ các quy tắc cơ bản.
5. **Bot Module (Mở rộng)**: Cuối cùng, thực hiện các thuật toán tìm nước đi cho Bot. Bắt đầu từ mức `EASY` (ngẫu nhiên) trước khi thử thách với các thuật toán thông minh hơn.

Lộ trình này giúp bạn luôn có một phiên bản chương trình "chạy được" (*runnable*) ở mỗi giai đoạn, tránh việc viết quá nhiều code mà không thể kiểm thử ngay.

Lưu ý rằng tài liệu này **không cung cấp lời giải hoàn chỉnh**. Các bạn cần dựa trên các khung hàm có sẵn để tự thiết kế thuật toán xử lý và tổ chức logic chương trình một cách tối ưu.

VII. Hướng dẫn thực nghiệm (Experiment Instructions)

Sau khi hoàn thành việc cài đặt, các bạn thực hiện quy trình kiểm thử để đảm bảo chương trình vận hành chính xác theo yêu cầu. Các kết quả thu được dưới đây bắt buộc phải có trong **báo cáo cuối cùng**.

VII.1 Biên dịch chương trình

Mã nguồn cài đặt cuối cùng nên được để ra một file riêng với tên `game.cpp`, hoặc tuỳ theo tên gọi mà các bạn muốn đặt. Chương trình yêu cầu trình biên dịch `g++` hỗ trợ chuẩn **C++20**. Các bạn có thể biên dịch bằng dòng lệnh hoặc công cụ hỗ trợ trong VS Code:

- **Dòng lệnh (Terminal)**:

```
g++ -std=c++20 -Wall -Wextra game.cpp -o game.exe
```

- **VS Code**: Sử dụng tổ hợp phím `Ctrl + Shift + B` (để biên dịch theo cách này, cần chuẩn bị file cấu hình `tasks.json`).

VII.2 Chạy chế độ tương tác (Interactive Mode)

Đây là chế độ mặc định khi chạy file thực thi với lời gọi (giả sử object có tên là `game.exe`):

```
./game.exe
```

Các bạn cần chụp ảnh màn hình terminal minh họa cho các giai đoạn sau:

- **Thiết lập**: Menu chọn `gameSetup` (kích thước, goal, chế độ chơi).
- **Diễn biến**: Hiển thị bàn cờ, thông báo lượt đi (`player move`) và quá trình Bot suy nghĩ (`bot move`).
- **Kết thúc**: Thông báo người thắng cuộc và tổng số lượt đi.

Lưu ý: *Nên thử nghiệm với các cấu hình khác nhau như* 3×3 *(goal 3) hoặc* 5×5 *(goal 4) để kiểm tra tính linh hoạt.*

VII.3 Chế độ Judge Mode

Chế độ này được sử dụng để kiểm thử tự động với dữ liệu đầu vào từ file. Kích hoạt bằng flag `--judge` hoặc `-j`:

```
./game.exe --judge --input input.txt
```

Trong báo cáo, các bạn cần cung cấp ảnh chụp terminal lời gọi lệnh trên. Đầu ra phải tuân thủ định dạng nghiêm ngặt: `<winner> <turns>` (Ví dụ: `0 7` cho Player 1 thắng sau 7 lượt).

VII.4 Chạy Test Script (Grader)

Để kiểm tra độ chính xác một cách toàn diện, các bạn sử dụng script `grader.py` đi kèm:

```
python grader.py --target game
```

Kết quả ghi nhận trong báo cáo phải bao gồm:

- Tổng số lượng testcase đã chạy.
- Số lượng testcase vượt qua (PASSED).
- Tỷ lệ chính xác (% Accuracy).

Lưu ý: *Kết quả ví dụ như: "10/10 testcases passed - Accuracy: 100.00%".*

VII.5 Ghi nhận kết quả vào báo cáo

Các bạn tổng hợp các minh chứng sau vào mục `./results/`:

1. **Ảnh chụp màn hình**: Các giai đoạn tương tác và lệnh chạy Judge Mode. Gợi ý đặt tên: `menu.png`, `display_board.png`, `player_move.png`, `bot_move.png`, ...
2. **Báo cáo từ Grader**: Bảng thống kê Pass/Fail từ script kiểm thử. Có thể copy kết quả chạy hoặc trình bày gọn lại theo phong cách cá nhân.
3. **File Log**: Copy file log với tên `log.txt` vào folder. Trích dẫn một đoạn log từ `game.log` (đặc biệt là các dòng `DEBUG` về thời gian thực thi của Bot) để minh chứng việc sử dụng `GameLogger` vào trong báo cáo (sẽ được nói ở phần sau).

Việc thực hiện đầy đủ các bước thực nghiệm trên không chỉ giúp các bạn đảm bảo chương trình chạy đúng theo yêu cầu kỹ thuật mà còn là cơ sở để đánh giá khả năng tối ưu hóa thuật toán (thông qua log thời gian thực thi).

Lưu ý: *Mọi kết quả trong báo cáo cần được trình bày trung thực, khớp với dữ liệu log và ảnh chụp màn hình thực tế từ máy tính cá nhân. Các trường hợp sai lệch dữ liệu sẽ bị coi là không hoàn thành yêu cầu thực nghiệm.*

VIII. Yêu cầu báo cáo (Report Requirements)

Báo cáo là thành phần bắt buộc, thể hiện tư duy thiết kế và quá trình thực nghiệm của các bạn. Lưu ý đây là bài tập **cá nhân**, các bạn phải tự hoàn thiện báo cáo riêng biệt.

VIII.1 Định dạng và Quy định chung

- **Hình thức**: Nộp file định dạng **PDF**. Giữ nguyên cấu trúc nội dung.
- **Công cụ**: Khuyến khích sử dụng LaTeX (Overleaf) hoặc MS Word/Google Docs nhưng phải xuất PDF.
- **Yêu cầu**: Nội dung báo cáo phải phản ánh chính xác mã nguồn đã nộp. Mọi hành vi sao chép sẽ bị xử lý theo quy định.
- **Ngôn ngữ**: Tiếng Việt, trích dẫn thuật ngữ Tiếng Anh (lần đầu nhắc đến).
- **Đặt tên**: lưu file báo cáo với định dạng tên sau:

  `report_level-1_<HoVaTen>_<mssv>.pdf`

  Ví dụ: `report_level-1_NguyenVanA_2502XXXX.pdf`

VIII.2 Cấu trúc nội dung (Outline)

Báo cáo cần tuân thủ đúng 06 mục sau đây:

1. Tổng quan (Overview)

- Thông tin cá nhân (Họ tên, MSSV).
- Danh sách các nhóm chức năng đã cài đặt thành công.
- Các cấp độ Bot đã triển khai (`EASY` là bắt buộc, `MEDIUM/HARD` là tùy chọn).

2. Chi tiết cài đặt (Implementation Details)

Phần này tập trung trả lời các vấn đề kỹ thuật cốt lõi. Sinh viên không cần trình bày toàn bộ mã nguồn mà tập trung vào:

- **Trả lời câu hỏi**: Trả lời các câu hỏi trọng tâm cho mỗi module (*Game Interaction, Game Renderer, Game Logic*) trình bày ở bên dưới.
- **Bot Module**: Trình bày chi tiết thuật toán cài đặt riêng cho từng level (Ví dụ: Cơ chế chọn ngẫu nhiên ở `EASY`, hàm lượng giá hoặc thuật toán tìm kiếm ở `MEDIUM/HARD`). Nếu có cài đặt, hãy so sánh tốc độ và sự thông minh giữa các level.

Câu hỏi yêu cầu:

1. **Game Interaction**: Bạn đã xử lý việc xác thực dữ liệu đầu vào (*input validation*) như thế nào? Trong trường hợp người dùng nhập ký tự không phải số hoặc tọa độ ngoài phạm vi, chương trình sẽ phản hồi ra sao để đảm bảo không bị treo (*crash*)?
2. **Game Renderer**: Hãy mô tả định dạng hiển thị bàn chơi mà bạn đã cài đặt. Làm thế nào để người chơi dễ dàng phân biệt giữa các hàng, cột và ký hiệu của hai bên (`X/O`) trên console?
3. **Game Logic**:
   - Viết mã giả (*pseudo-code*) cho thuật toán `checkWin(…)`: Bạn đã tối ưu việc quét các hướng (ngang, dọc, chéo) như thế nào để đảm bảo hiệu suất khi kích thước bàn cờ lớn?
   - Mô tả logic của hàm `checkDraw(…)`: Điều kiện chính xác để xác định một trận hòa là gì?
4. **Game Engine**: Hãy giải thích cách bạn điều phối lượt đi giữa **Người** và **Bot** trong vòng lặp chính. Làm thế nào để Engine nhận biết được trò chơi đã kết thúc để gọi hàm `endGame`?

5. **Sử dụng Game Helper**: Bạn đã sử dụng hàm `measureExecutionTime` như thế nào để đánh giá hiệu năng của Bot? Hãy trích dẫn một kết quả đo thực tế từ log và nhận xét về sự chênh lệch thời gian xử lý giữa các cấp độ Bot (nếu có).

3. Kết quả thực nghiệm (Experiment Results)

Cung cấp minh chứng trực quan về sự vận hành của chương trình:

- **Ảnh chụp màn hình**: Giao diện tương tác (Menu thiết lập, bàn chơi, thông báo nước đi).
- **Báo cáo Grader**: Chụp kết quả chạy từ file `grader.py` (Số lượng test pass, tỷ lệ % accuracy).
- **Trích dẫn Log**: Trích dẫn các đoạn log tiêu biểu từ `game.log` để chứng minh việc quản lý luồng dữ liệu và đo thời gian thực thi.

4. Nhận xét và tổng kết bài làm

Phần này là không gian để bản thân tự đúc kết kinh nghiệm và nhìn nhận lại quá trình thực hiện dự án. Hãy trả lời ngắn gọn các câu hỏi gợi ý sau:

- **Về phong cách lập trình**: Theo trải nghiệm của bạn, việc sử dụng phong cách lập trình thủ tục (*Procedural Programming*) có điểm mạnh và điểm yếu gì trong việc quản lý luồng trò chơi? Liệu cách tiếp cận này có dễ bảo trì khi trò chơi trở nên phức tạp hơn không?
- **Về giao diện người dùng**: Làm việc với *Terminal Interface* (dòng lệnh) mang lại những điều thú vị và khó khăn gì so với các ứng dụng có giao diện đồ họa (GUI) thông thường?
- **Về tổ chức mã nguồn**: Việc tập trung toàn bộ mã nguồn vào một file (*monolithic file*) dù đã có chú thích (*docstring/comment*) rõ ràng có thực sự hiệu quả cho việc phát triển lâu dài? Theo bạn, có cách tổ chức nào khác giúp chương trình chuyên nghiệp và dễ quản lý hơn không?
- **Bài học rút ra**: Khó khăn lớn nhất bạn đã vượt qua là gì (ví dụ: lỗi logic `checkWin`, xử lý nhập liệu, hay tối ưu thuật toán Bot)?

5. Ghi chú sử dụng AI (AI Disclosure)

Sinh viên cần trung thực khai báo việc sử dụng các công cụ AI (như ChatGPT, GitHub Copilot,...) trong quá trình làm bài:

- Mục đích sử dụng (Ví dụ: Giải thích lỗi, gợi ý cấu trúc code, sửa lỗi LaTeX).

- Mức độ đóng góp của AI vào sản phẩm cuối cùng.

Đọc kỹ phần **Chính sách sử dụng AI**, để nắm được các lưu ý quan trọng trước khi tiến hành sử dụng AI trong quá trình làm bài.

6. Nguồn tham khảo (References)

Liệt kê các tài liệu, trang web hoặc thư viện (nếu có) đã sử dụng trong quá trình tìm hiểu và thực hiện bài tập.

Báo cáo cần được trình bày rõ ràng, súc tích. Các hình ảnh minh họa phải sắc nét và có chú thích đầy đủ. Đây là cơ sở quan trọng để giảng viên đánh giá mức độ hiểu bài và kỹ năng triển khai thực tế của các bạn.

IX. Chính sách sử dụng công cụ AI (AI Usage Policy)

Trong khóa học này, các bạn **được phép sử dụng các công cụ AI** (ví dụ: ChatGPT, Copilot, Claude, Gemini, v.v.) như một công cụ hỗ trợ học tập. Tuy nhiên, việc sử dụng AI cần tuân thủ các nguyên tắc sau.

IX.1 Nguyên tắc chung

- AI chỉ nên được sử dụng như **công cụ hỗ trợ học tập**, không phải để thay thế hoàn toàn quá trình suy nghĩ của các bạn.
- Sinh viên cần **hiểu rõ mã nguồn** mà mình nộp.
- Sinh viên phải có khả năng **giải thích cách hoạt động của chương trình**.

Trong quá trình chấm bài hoặc vấn đáp, nếu các bạn không thể giải thích mã nguồn của mình, bài làm có thể bị đánh giá lại.

IX.2 Yêu cầu khai báo

Nếu các bạn sử dụng AI trong quá trình làm bài, cần khai báo rõ trong báo cáo. Việc khai báo này giúp đảm bảo tính minh bạch trong học tập.

Thông tin khai báo nên bao gồm:

- Tên công cụ AI đã sử dụng
- Mục đích sử dụng (nên viết chi tiết)

- Mức độ sử dụng (ước lượng)

Ví dụ:

```
AI Tool: ChatGPT
Purpose: gợi ý ý tưởng cho hàm checkWin()
Estimated usage: ~20%
```

IX.3 Những việc không được phép

Sinh viên **không được phép**:

- Sao chép toàn bộ mã nguồn do AI sinh ra mà không hiểu nội dung
- Nộp bài giống nhau giữa các nhóm
- Sử dụng AI để vượt qua hệ thống kiểm tra mà không thực sự làm bài

Nếu phát hiện hành vi vi phạm học thuật, bài làm có thể bị xử lý theo quy định của môn học và của nhà trường.

IX.4 Mục tiêu của chính sách

Mục tiêu của chính sách này không phải là cấm AI, mà là khuyến khích các bạn:

- sử dụng AI một cách có trách nhiệm
- hiểu rõ mã nguồn của mình
- phát triển kỹ năng lập trình và tư duy giải quyết vấn đề

Đội ngũ trợ giảng thực hành môn học chúng mình khuyến khích các bạn làm bài tập này bằng đúng khả năng của bản thân. Dù kết quả có như thế nào thì đây cũng là một trải nghiệm đáng được dành thời gian và tâm sức của mình vào.

Công nghệ AI là một bước tiến lớn giúp thay đổi cách chúng ta học lập trình, nhưng kỹ năng tư duy logic và khả năng làm chủ mã nguồn mới là giá trị cốt lõi đi cùng các bạn trong sự nghiệp. Hãy coi AI như một người cộng sự thông minh để trao đổi ý tưởng, chứ đừng biến mình thành "người vận chuyển" mã nguồn từ máy tính vào bài tập.

Chúc các bạn có một hành trình trải nghiệm thú vị, học hỏi được nhiều điều mới và tự hào về sản phẩm do chính tay mình hoàn thiện!

X. Nộp bài (Submission)

Sau đây là hướng dẫn chuẩn bị cho phần nộp bài. Yêu cầu các bạn đọc kỹ để nộp bài đúng yêu cầu.

X.1 Các thành phần cần nộp

Bài nộp cần bao gồm các thành phần sau:

- **Mã nguồn chương trình** (file `.cpp`)
- **Báo cáo PDF** (file `.pdf`)
- **Thư mục kết quả thực nghiệm** (folder `results/`)

Toàn bộ các file cần được đặt trong một thư mục duy nhất trước khi nén.

X.2 Cấu trúc thư mục

Các bạn nên tổ chức thư mục nộp bài theo cấu trúc sau:

```
proeject_level_1/
│
├── results/
│   ├── menu.png
│   ├── display_board.png
│   ├── player_move.png
│   ├── bot_move.png
│   ├── ...
│   └── log.txt
│
├── game.cpp
└── report_level-1_<HoVaTen>_<MSSV>.pdf
```

Trong đó:

- `src/` chứa toàn bộ mã nguồn
- `report/` chứa báo cáo PDF
- `logs/` chứa file log được tạo ra khi chạy chương trình
- `README.txt` mô tả ngắn gọn cách biên dịch và chạy chương trình

X.3 Định dạng nộp bài

Thư mục bài làm cần được nén thành một file:

```
project_level-1_<HoVaTen>_<mssv>.zip
```

Ví dụ:

```
proeject_level-1_NguyenVanA_2502XXXX.zip
```

Chỉ nộp **một file nén duy nhất**.

X.4 Thời hạn nộp bài

Thời hạn nộp bài sẽ được thông báo trên hệ thống của môn học.

Các bạn nên nộp bài trước hạn để tránh các vấn đề kỹ thuật vào thời điểm gần deadline.

X.5 Lưu ý

- Đảm bảo chương trình có thể biên dịch và chạy được, trong trường hợp có nhiều file mã nguồn, cần chỉ rõ cách biên dịch;
- Kiểm tra lại file nén trước khi nộp;
- Không nộp các file không cần thiết (file object, file tạm, v.v.).